R-02: SetPassword Token Security Review

Scope and conclusion

This review covers only the administrator-created student/teacher password setup workflow and `AccountController.SetPassword`. The repository confirms that the setup link carries a password-reset token in its query string, so the link is a bearer credential. I found no demonstrated application-level disclosure or token-validation defect that justifies changing the code. Production request logging, browser/device handling, and link-sharing practices are outside this repository and remain unverified.

Workflow and code locations

1. `Areas/Admin/Controllers/StudentsController.cs` creates a passwordless student with `UserManager.CreateAsync(student)`, assigns the Student role, commits, then calls `GeneratePasswordResetTokenAsync(student)`. It places the generated link in the TempData key `AccountSetupUrl` and the created username in TempData.
2. `Areas/Admin/Controllers/TeachersController.cs` creates a passwordless teacher, role, profile, and subject links, commits, then generates the same kind of token and stores the setup link in TempData.
3. `Areas/Admin/Views/Students/Index.cshtml` and `Areas/Admin/Views/Teachers/Index.cshtml` render the link for an authenticated administrator to open and manually share. Both state it is not sent by email.
4. `Controllers/AccountController.cs` exposes an anonymous GET at `SetPassword(userId, token)`. It looks up the specified user and returns 404 if the account does not exist or already has a password. Otherwise it places the user ID and token into `SetPasswordViewModel`.
5. `Views/Account/SetPassword.cshtml` carries both fields in hidden inputs on an antiforgery-protected POST. The POST looks up that user again, rejects a missing/already-passworded account, then calls `ResetPasswordAsync(user, model.Token, model.Password)`. Successful setup renders `SetPasswordComplete`.

Token behavior and findings

Question	Finding
Provider	`Program.cs` calls `.AddDefaultTokenProviders()`. `GeneratePasswordResetTokenAsync` uses Identity's configured password-reset provider, which the default provider registration maps to the Data Protection token provider.
Effective lifetime	No override is configured in `Program.cs` or repository settings. ASP.NET Core Identity's `DataProtectionTokenProviderOptions.TokenLifespan` default is one day. This is framework behavior, verified against Microsoft documentation and the Identity API reference.
User and purpose binding	`ResetPasswordAsync` validates the token through the password-reset token provider for the `user` instance passed to it. The Data Protection token includes the user ID, purpose, issue time, and (when supported) security stamp. A token for a different user or purpose does not validate. See Identity's provider implementation and UserManager implementation.
Reuse after success	The POST also rejects users with a non-empty `PasswordHash`. In addition, successful Identity password reset updates the password and security stamp for the user store, so the previous token is no longer valid. This is a one-time outcome in the current workflow.
Invalid/expired token	`ResetPasswordAsync` returns a failed `IdentityResult`; the controller adds its error descriptions to ModelState and redisplays the setup form. No password is set on that failed result.
Missing user	Both GET and POST return 404 for a missing user; already-passworded users also receive 404.
App-level disclosure	The repository contains no explicit log statement that writes the setup URL or token. The controller displays Identity error descriptions, not the submitted token. The setup view references local CSS/JavaScript and has no external resource URL. The token is intentionally present in the generated URL, the administrator page, and the setup form's hidden field.
Resend/recovery	No regeneration or resend mechanism exists. A newly generated link can be made only by an administrator workflow that creates a new account; there is no implemented recovery action for an existing passwordless account. Adding that workflow would exceed the requested scope and is not needed to establish a token-validation defect.

Confirmed weaknesses and unverified risks

Confirmed design exposure: the token is a bearer credential in a query-string URL and is displayed to an administrator for manual sharing. Anyone who obtains a still-valid link can attempt to set the intended account's password. This follows directly from the implementation and is inherent in this setup-link flow; the token is time-limited, user/purpose-bound, and invalidated after successful password setup.

Not confirmed from source: whether production proxy/web-server/APM logs retain query strings, whether the link is copied to an unintended recipient, or how endpoint/browser history is protected. The

app source has no request-logging configuration specific to this route. No deployment or infrastructure configuration was available to inspect. Do not infer that these external systems leak tokens from the mere fact that a query parameter is used.

No additional confirmed code defect: the GET is anonymous by design so the invited user can reach the setup page, while the POST is antiforgery-protected and Identity validates the token for the specified account. The public GET by itself does not set a password. Token validation is delegated to Identity, and both actions check the account state.

Root cause and change decision

The setup credential must be delivered in the manually shared setup link. Query-string placement makes it visible to browser URL/history and potentially to infrastructure that records request targets. The repository does not establish that the token is logged or exposed to an unintended party, and no delivery, logging, or URL-handling requirement was supplied that would select a safe alternative. Removing the token from the link would require a different delivery/state mechanism and would change the workflow. Therefore no code change was made; selecting a speculative redesign would not be a minimal justified fix.

Files changed

- `R02_SETPASSWORD_TOKEN_SECURITY_FIX.md` — added this R-02-only investigation and verification record.
- Application code and configuration — unchanged because no confirmed defect with a repository-supported targeted fix was identified.

Verification

- Source inspection traced both admin account creation flows, Identity token generation/validation calls, `SetPasswordViewModel`, setup views, and authentication cookie configuration relevant to the flow.
- Framework token lifetime/provider behavior was checked against Microsoft documentation and ASP.NET Core Identity source.
- `dotnet build Manasa.csproj --no-restore` — succeeded with 0 warnings and 0 errors.
- No test project or existing focused tests were found in this checkout. Invalid, expired, wrong-user, and successful setup scenarios were not executed. No database migration or destructive operation was run.

Remaining concerns and next steps

- Treat setup links as secrets: share only with the intended user over an approved channel and avoid recording or forwarding the full URL.
- Verify that production reverse proxies, web servers, and observability tools redact query strings for the setup endpoint, and review their retention/access controls.
- If policy requires shorter validity or reliable recovery, define those requirements before changing token lifetime or adding an administrator reissue flow.